discopt: A Differentiable Mixed-Integer Nonlinear Programming Solver with a Memory-Safe Interior Point Method

John R. Kitchin

2026-02-14

1 Abstract

We present discopt, a hybrid Rust/JAX/Python solver for mixed-integer nonlinear programming (MINLP) problems, together with ripopt, a faithful Rust translation of Ipopt's interior point algorithm. The system makes five contributions: (1) differentiable MINLP solving via implicit differentiation of the KKT system, enabling decision-focused learning through nonconvex optimization layers; (2) batched node evaluation using `jax.vmap` for data-parallel branch-and-bound; (3) a pure-JAX interior point method with Mehrotra predictor-corrector steps and Ipopt-style filter line search that is JIT-compilable and vmappable; (4) ripopt, a memory-safe Rust IPM with zero C/Fortran dependencies that implements Ipopt's numerical hardening through a source-level translation; and (5) a compositional relaxation compiler supporting McCormick, α BB, piecewise McCormick, specialized envelopes, and ICNN-learned relaxations. The solver includes an expression DAG convexity detector that enables adaptive dispatch (skipping relaxation overhead for convex problems), per-constraint outer approximation cuts, reliability branching with pseudocost tracking and LP-based strong branching, and OBBT with incumbent cutoff for progressive bound tightening. All three NLP backends (pure-JAX IPM, ripopt, Ipopt) produce identical optimal solutions on a suite of 9 NLP and 24 MINLP benchmark problems with zero incorrect results. On a decision-focused learning task, differentiating through the solver achieves 36% regret reduction versus two-stage predict-then-optimize. The algebraic coefficient extraction path provides 17–226× speedups for LP and QP subproblems. Source code is available under EPL-2.0 license (discopt and ripopt).

2 Introduction

Mixed-integer nonlinear programming (MINLP) is a powerful framework for optimization problems involving both discrete choices and nonlinear relationships, with applications spanning process synthesis, network design, scheduling, and machine learning pipeline optimization.[1,2] The dominant algorithmic paradigm for deterministic global MINLP is spatial branch

and bound (sBB), which enumerates a search tree of progressively tighter variable-bound regions, solving a convex relaxation at each node.[3–5]

State-of-the-art MINLP solvers—BARON,[6] Couenne, SCIP,[7] and Bonmin[8] are monolithic C, C++, or Fortran codebases that have accumulated decades of numerical hardening. While effective, this architecture creates three gaps in the current solver ecosystem.

This paper describes discopt and ripopt, which together address these three gaps. The contributions are:

1. **Differentiable MINLP** via two mechanisms: the envelope theorem (Level 1) for fast objective sensitivity, and implicit differentiation of the KKT system (Level 3) for full primal–dual sensitivity matrices, enabling decision-focused learning through nonconvex solvers (\S 6).

2. **Batched node evaluation** using `jax.vmap` over NLP relaxation solves at B&B nodes, with warm-starting from parent solutions (\S 4.1).

3. **Pure-JAX interior point method** with Mehrotra predictor-corrector,[9] Cholesky-based inertia correction, Ipopt-style filter line search,[10] and carry-based `jax.lax.while_loop` for JIT cache reuse (\S 4.1).

4. **ripopt**: a source-level C++ $\rightarrow$ Rust translation of Ipopt's core IPM algorithm, producing a standalone memory-safe NLP solver with a clean `NlpProblem` trait interface (\S 4.2).

5. **Compositional relaxation framework** with 19 McCormick relaxation functions,[11,12] α BB underestimators,[13,14] piecewise McCormick with adaptive partitioning[15] achieving 91–94% gap reduction, and ICNN-learned relaxations[16] (\S 5).

The remainder of this paper is organized as follows. Section 2 reviews background and related work. Section 3 describes the discopt architecture. Section 4 presents the three NLP solver backends. Section 5 details the convex relaxation framework. Section 6 covers differentiable optimization. Section 7 reports computational experiments. Section 8 discusses limitations and future work, and Section 9 concludes.

3 Background and Related Work

3.1 Spatial Branch and Bound for MINLP

Spatial branch and bound (sBB) is the dominant paradigm for deterministic global optimization of MINLP problems.[1] The algorithm maintains a search tree where each node represents a subproblem defined by tightened variable bounds. At each node, a convex relaxation of the original nonconvex problem is solved; the relaxation's optimal value provides a valid lower bound on the global optimum within that region.

The framework was introduced by Land and Doig[3] for integer programming and extended by Dakin.[4] Smith and Pantelides[5] formalized spatial B&B for nonconvex MINLP by combining variable-bound branching with factorable relaxation construction.

Modern sBB solvers enhance the basic framework with bound tightening techniques—feasibility-based bound tightening (FBBT) and optimization-based bound tightening (OBBT)[17,18] — which propagate bound improvements through the expression graph, and primal heuristics such as the feasibility pump[19] that attempt to find good incumbent solutions early in the search.

3.2 Convex Relaxations

The quality of the convex relaxation at each node directly determines the effectiveness of the B&B search. Weaker relaxations produce loose lower bounds, leading to more nodes explored before convergence.

McCormick relaxations[11] construct convex and concave envelopes for factorable functions by composing relaxations of elementary operations (addition, multiplication, univariate functions). Tsoukalas and Mitsos[12] extended this to the multivariate case. The α BB method[13,14] adds a separable quadratic perturbation to ensure convexity, where the perturbation magnitude α is determined by the minimum eigenvalue (or Gershgorin estimate) of the objective Hessian.

Piecewise McCormick relaxations[15] partition the variable domain into subintervals and construct tighter McCormick envelopes on each partition, significantly reducing the relaxation gap at the cost of additional auxiliary variables. Outer approximation (OA)[20,21] and reformulation-linearization technique (RLT)[22] provide complementary cutting-plane approaches.

3.3 Interior Point Methods for NLP

The continuous NLP relaxation at each B&B node is typically solved by an interior point method (IPM). The primal-dual IPM[23,24] transforms inequality constraints into barrier terms in the objective, then applies Newton's method to the resulting KKT system. Key algorithmic components include the Mehrotra predictor-corrector step,[9] which adaptively adjusts the centering parameter for faster convergence, and filter line search,[10,25] which accepts steps that improve either the objective or constraint feasibility.

Ipopt[10] is the most widely used open-source NLP solver, implementing a filter line-search interior point method with inertia correction, feasibility restoration, and warm-starting. It has accumulated over 20 years of numerical hardening for edge cases including degenerate Jacobians, rank-deficient KKT systems, and cycling detection.

Recent work on JAX-native optimization includes MPAX,[26] a first-order solver for large-scale LP and QP that implements restarted primal-dual hybrid gradient (PDHG) variants: r2HPDHG for LP and rAPDHG for QP, with Ruiz and Pock-Chambolle diagonal preconditioning, adaptive restarts, and infeasibility detection. MPAX leverages JAX for hardware portability (CPU/GPU/TPU), batched solving via `vmap`, multi-GPU distribution via `pmap`, and automatic differentiation through solutions. The two systems occupy complementary niches: MPAX's first-order iterations are cheap (one matrix-vector product each) but converge slowly ($O(1/k)$ to $O(1/k^2)$ with restarts), making it well-suited for very large sparse LP/QP instances ($n > 10^5$) where moderate accuracy suffices. discopt's second-order IPM performs expensive Cholesky factorizations ($O(n^3)$) but converges quadratically, targeting

the small-to-medium nonconvex NLP subproblems ($n < 10^4$) that arise at each B&B node where high-accuracy solutions are needed for valid dual bounds. Unlike MPAX, discopt handles nonconvex objectives (via inertia correction), nonlinear constraints (via augmented KKT), and integer variables (via spatial B&B). JAX device placement in discopt is controlled by the `JAX_PLATFORMS` environment variable, inheriting GPU/TPU acceleration automatically when a suitable backend is available.

Sensitivity analysis of NLP solutions with respect to problem parameters is well-established.[27] Pirnay et al.[28] implemented the sIPOPT framework within Ipopt for efficient parametric sensitivity computation by reusing the KKT factorization.

3.4 Differentiable Optimization

The idea of differentiating through an optimization solver to enable end-to-end learning has received significant recent attention. Amos and Kolter[29] introduced OptNet for differentiating through quadratic programs. Agrawal et al.[30] extended this to general convex programs via cvxpylayers. Elmachtoub and Grigas[31] formalized the "smart predict, then optimize" framework, and Wilder et al.[32] and Mandi et al.[33] developed methods for combinatorial optimization with decision-focused learning.

A critical gap remains: **none of these approaches handle nonconvex MINLP**. Existing differentiable optimization layers require convexity (OptNet, cvxpylayers) or use surrogate loss functions that approximate but do not exactly differentiate through the solver (decision-focused learning for combinatorics). discopt fills this gap by providing exact gradients through a nonconvex NLP solver via implicit differentiation of the KKT conditions.

3.5 Gaps in the State of the Art

Table 1 summarizes the capabilities of existing systems versus discopt.

Feature	BARON	SCIP	Bonmin	cvxpylayers	MPAX	discopt
Nonconvex MINLP	Yes	Yes	Yes	No	No	Yes
Nonlinear constraints	Yes	Yes	Yes	No	No	Yes
Differentiable solve	No	No	No	Yes (convex)	Yes (LP/QP)	Yes
Batched solving	No	No	No	N/A	Yes	Yes
GPU/TPU support	No	No	No	N/A	Yes	Yes
Memory-safe solver	No	No	No	N/A	Yes	Yes
Zero C/Fortran deps	No	No	No	Yes	Yes	Yes
DFL through MINLP	No	No	No	No	No	Yes
Compositional relaxation	Yes	Partial	No	N/A	N/A	Yes

Table 1: Comparison of discopt with existing optimization frameworks. DFL = decision-focused learning. GPU/TPU support for discopt is via JAX backend selection (`JAX_PLATFORMS`).

4 The discopt Architecture

4.1 System Overview

discopt is organized as a three-layer system: (1) a Python modeling API for problem formulation, (2) a JAX-based numerical layer for evaluation, automatic differentiation, and batch computation, and (3) a Rust backend for tree management, expression structure detection, and presolve.

```
User Model (Python)
     |
     v
Expression DAG ---> DAG Compiler ---> JAX Functions (JIT/grad/vmap)
     |                                        |
     v                                        v
Rust Expression IR             NLP Evaluator (obj, grad, Hess, Jac)
     |                                        |
     v                                        v
B&B Tree Manager               NLP Solver Backend
(node pool, branching,         (IPM / ripopt / Ipopt)
 bound tightening)                     |
     |                                 v
     +-------- batch dispatch --------+
                    |
                    v
             SolveResult
```

An automatic problem classifier inspects the expression DAG to categorize problems as LP, QP, NLP, MILP, MIQP, or MINLP. For LP and QP subclasses, discopt dispatches to specialized IPM solvers that extract algebraic coefficients directly from the DAG, bypassing JAX tracing and autodiff entirely. This algebraic extraction path provides 100–226× speedups over the generic NLP path (\S 7.4).

4.2 Modeling API

The modeling API uses operator overloading to construct an expression DAG that maps to both JAX-traceable functions and Rust expression IR:

```python
import discopt.modeling as dm

m = dm.Model("portfolio")
x = m.continuous("weights", shape=(n,), lb=0, ub=1)
y = m.binary("selected", shape=(n,))

m.minimize(risk @ (x * x) - expected_return @ x)
m.subject_to(dm.sum(x) == 1, name="budget")
m.subject_to(x <= y, name="indicator")  # x_i > 0 => y_i = 1
m.subject_to(dm.sum(y) <= k, name="cardinality")

result = m.solve()
```

The API supports continuous, binary, and integer variables; parameters (for sensitivity analysis and decision-focused learning); generalized disjunctive programming (GDP) constraints with automatic big-M reformulation; and standard mathematical functions (`exp`, `log`, `sin`, `cos`, `sqrt`, `abs`, and matrix operations).

4.3 JAX DAG Compiler

The DAG compiler walks the expression tree and produces pure JAX functions compatible with `jax.jit`, `jax.grad`, and `jax.vmap`. Each expression node type (`Variable`, `Constant`, `BinaryOp`, `FunctionCall`, etc.) maps to a corresponding JAX operation. The compiler handles:

- **Lazy evaluation**: expressions are not evaluated until the DAG is compiled
- **Automatic differentiation**: the JAX function graph supports forward and reverse mode AD for gradients, Hessians, and Jacobians
- **Vectorization**: `jax.vmap` can map the compiled function over batches of variable values or bound vectors
- **JIT compilation**: compiled functions are traced and optimized by XLA for efficient repeated evaluation

4.4 Rust Backend

The Rust backend (`discopt-core` and `discopt-python` crates) provides three components: (1) an expression IR that mirrors the Python DAG, (2) a B&B tree manager with node pool, branching, and pseudocost tracking, and (3) presolve routines including FBBT, probing, and simplification. All Rust data structures are exposed to Python via PyO3 bindings[34] with zero-copy numpy array transfer.

4.4.1 Reliability Branching

Variable selection in B&B significantly impacts solver performance. discopt implements reliability branching,[35] which combines pseudocost branching with strong branching in a hybrid scheme.

After each branching decision, the Rust `TreeManager` records the bound improvement per unit of variable change (pseudocost) for both up and down branches. Running averages are maintained per variable. When selecting a branching variable, the algorithm examines all fractional integer variables and:

1. For variables with $\geq \eta_{\text{rel}}$ (default 8) pseudocost observations: scores are computed as the product $(10^{-6}+d_i^-)(10^{-6}+d_i^+)$ where $d_i^\pm$ are the pseudocost estimates for down/up branching.
2. For variables with $< \eta_{\text{rel}}$ observations: Python-side strong branching via LP relaxation solves with HiGHS.[36] For each candidate, two LP subproblems are constructed by linearizing the NLP at the current solution (gradient as objective, Jacobian-based constraint linearization), one with the variable's upper bound fixed to $\lfloor x_i \rfloor$ (down) and one with the lower bound fixed to $\lceil x_i \rceil$ (up). The resulting LP bound improvements determine the score.

3. Branch hints are passed from Python back to the Rust `TreeManager` via `set_branch_hints()`, where they override the default most-fractional selection.

As the tree matures and pseudocost observations accumulate, the algorithm naturally transitions from expensive strong branching to cheap pseudocost branching, achieving near-optimal variable selection at amortized low cost.

4.4.2 OBBT with Incumbent Cutoff

Optimality-based bound tightening (OBBT)[18] solves LP relaxations to find the tightest possible bounds on each variable: $\underline{x}_i = \min_x x_i$ and $\overline{x}_i = \max_x x_i$ subject to the model's linear constraints and variable bounds.

When an incumbent solution with objective value z^* is known, discopt adds the constraint $c^T x \leq z^*$ (using linear objective coefficients) to the OBBT LPs. This often dramatically tightens bounds because many variable values become infeasible when the objective must improve upon the incumbent. For maximization problems, the constraint is negated: $-c^T x \leq -z^*$.

OBBT is invoked periodically during B&B whenever a new incumbent is found and the problem has ≤ 200 variables. Each LP is solved with HiGHS using warm-starting from the previous LP's basis, keeping the per-variable LP solve cost low. Tightened bounds propagate to all open nodes, potentially enabling additional pruning and tighter relaxations at subsequent nodes.

4.4.3 FBBT with Incumbent Cutoff

Feasibility-based bound tightening (FBBT)[17,37] propagates bound information through the expression DAG without solving auxiliary optimization problems, making it much cheaper than OBBT. Each constraint $g(x) \leq 0$ induces range implications: if a variable's contribution to g exceeds the slack, its domain can be tightened.

When an incumbent with objective value z^* is available, discopt adds the cutoff constraint $f(x) \leq z^*$ to the FBBT propagation. This is implemented in the Rust `fbbt_with_cutoff` function, which takes the expression IR, current variable bounds, and the incumbent objective. The cutoff often triggers cascading domain reductions: if reducing one variable's range via the objective cutoff renders a constraint infeasible for extreme values of another variable, that variable's bounds tighten as well.

FBBT with incumbent cutoff runs at every B&B node (before the NLP relaxation solve), while OBBT is reserved for periodic invocation when a new incumbent is found. The combination provides complementary bound tightening: FBBT is fast but conservative; OBBT is expensive but finds the tightest LP-based bounds.

5 NLP Solver Backends

discopt supports three interchangeable NLP solver backends, selectable via `Model.solve(nlp_solver...)`=. All three implement the same interface: given an objective function $f(x)$, constraints $g(x)$,

variable bounds $x_l \leq x \leq x_u$, and constraint bounds $g_l \leq g(x) \leq g_u$, find x^* that minimizes f subject to all constraints.

5.1 Pure-JAX Interior Point Method

:PROPERTIES: :CUSTOM$_{\text{ID}}$: sec-jax-ipm :END:

The pure-JAX IPM implements a primal-dual barrier method that is fully JIT-compilable and vmappable. The algorithm solves the augmented KKT system:

$$\begin{bmatrix} H + \Sigma + \delta_w I & J^T \\ J & -D - \delta_c I \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} r_x \\ r_y \end{bmatrix} \tag{1}$$

where $H = \nabla^2_{xx}\mathcal{L}$ is the Hessian of the Lagrangian, $\Sigma = \text{diag}(z_l/s_l + z_u/s_u)$ is the barrier Hessian diagonal, J is the constraint Jacobian, D is the condensed slack diagonal for inequality constraints, and δ_w, δ_c are inertia correction regularizations.

5.1.1 Mehrotra Predictor-Corrector

Following Mehrotra,[9] each iteration consists of two linear system solves:

1. **Affine predictor** ($\mu = 0$): solve the KKT system with zero centering to obtain $(\Delta x^{\text{aff}}, \Delta y^{\text{aff}})$. Compute the affine complementarity μ^{aff}.
2. **Centering parameter**: $\sigma = (\mu^{\text{aff}}/\mu^{\text{curr}})^3$, clipped to $[0, 1]$.
3. **Corrector**: re-solve with centering $\sigma\mu$ and cross-product correction terms $\Delta s_i \Delta z_i / s_i$.

The corrector step provides second-order information about the central path, typically reducing the iteration count by 30–50\% compared to a single Newton step.

5.1.2 Inertia Correction

The KKT system must have correct inertia (n positive, m negative eigenvalues) for the Newton direction to be a descent direction. We detect incorrect inertia via Cholesky factorization: if the upper-left block $W = H + \Sigma + \delta_w I$ produces NaN in its Cholesky factor, the inertia is incorrect. The regularization δ_w is increased by a factor of 8 (up to $\delta_w^{\max} = 10^{40}$) until correct inertia is achieved. This Cholesky-based detection is approximately 3× faster than eigenvalue decomposition.

5.1.3 Filter Line Search

Step acceptance follows Ipopt's filter line search.[10] The algorithm maintains an ℓ_1 merit function $\phi(x) = f(x) + \nu\|c(x)\|_1$ with adaptive penalty parameter ν and performs backtracking with Armijo-type sufficient decrease. A stall detection mechanism triggers projected steepest-descent recovery after 3 consecutive line search failures, handling the Maratos effect near active bound constraints.

5.1.4 Carry-Based `while_loop`

A critical implementation detail for JAX performance: all problem data (bounds, constraint bounds, masks) is placed in the `while_loop` carry state as a `NamedTuple`, rather than captured in the closure. Closure-captured data forces JIT retracing for each new problem instance, while carry-based data allows JIT cache reuse. Problem dimensions are derived from array shapes (`c.shape[0]`, `b.shape[0]`) rather than stored as integers, since integers in the carry become JAX tracers.

5.1.5 Batch Solving via `jax.vmap`

The `solve_nlp_batch` function applies `jax.vmap` over the initial point and variable bound dimensions:

```python
def solve_nlp_batch(obj_fn, con_fn, x0_batch, xl_batch, xu_batch, g_l, g_u, opts):
    def _solve_single(x0, xl, xu):
        return ipm_solve(obj_fn, con_fn, x0, xl, xu, g_l, g_u, opts)
    return jax.vmap(_solve_single)(x0_batch, xl_batch, xu_batch)
```

This solves all k node relaxations simultaneously, providing approximately k-fold parallelism on hardware with sufficient FLOP throughput. The objective and constraint functions are shared across the batch; only the starting points and variable bounds vary per instance.

At the root node of the B&B tree, multi-start solving uses 5 diverse starting points (midpoint, lower-quarter, upper-quarter, and 2 random) solved simultaneously via `vmap`. On non-root nodes, the solver warm-starts from the parent node's solution clipped to the child's tightened bounds.

5.2 ripopt: A Memory-Safe IPM in Rust

:PROPERTIES: :CUSTOM$_{\text{ID}}$: sec-ripopt :END:

ripopt (Rust Interior Point OPTimizer) is a source-level translation of Ipopt's core interior point algorithm from C++ to Rust. It is **not** a wrapper around C++ Ipopt; it is a standalone Rust crate that produces native code with Rust's ownership and borrowing guarantees.

5.2.1 Motivation

Three considerations drove the decision to translate rather than wrap:

1. **Numerical hardening.** Ipopt handles hundreds of edge cases accumulated over two decades: degenerate Jacobians, rank-deficient KKT systems, cycling detection, and tiny-step recovery. A from-scratch IPM would rediscover these failures one benchmark at a time. Translation preserves this institutional knowledge.

2. **Memory safety.** Rust's ownership model eliminates use-after-free bugs, data races, and buffer overflows at compile time—categories of bugs that have affected C++ numerical codes. The `cargo clippy` linter and optional `miri` interpreter catch additional issues.

3. **Deployment simplicity.** ripopt compiles with `cargo build` with zero C or Fortran dependencies. The entire solver is a single Rust crate, eliminating the MUMPS/BLAS/LAPACK dependency chain that makes Ipopt installation non-trivial.

5.2.2 Architecture

ripopt exposes a clean `NlpProblem` trait that users implement:

```rust
pub trait NlpProblem {
    fn num_variables(&self) -> usize;
    fn num_constraints(&self) -> usize;
    fn bounds(&self, x_l: &mut [f64], x_u: &mut [f64]);
    fn constraint_bounds(&self, g_l: &mut [f64], g_u: &mut [f64]);
    fn initial_point(&self, x0: &mut [f64]);
    fn objective(&self, x: &[f64]) -> f64;
    fn gradient(&self, x: &[f64], grad: &mut [f64]);
    fn constraints(&self, x: &[f64], g: &mut [f64]);
    fn jacobian_structure(&self) -> (Vec<usize>, Vec<usize>);
    fn jacobian_values(&self, x: &[f64], vals: &mut [f64]);
    fn hessian_structure(&self) -> (Vec<usize>, Vec<usize>);
    fn hessian_values(&self, x: &[f64], obj_factor: f64,
                      lambda: &[f64], vals: &mut [f64]);
}
```

The solver implements filter line search with Ipopt-style acceptance criteria, inertia correction via the `LinearSolver` trait (pluggable backends), feasibility restoration, warm-starting from previous solutions, and configurable convergence criteria.

5.2.3 PyO3 Integration

The `discopt-python` crate wraps ripopt with PyO3 bindings.[34] A `PyNlpProblem` struct implements the `NlpProblem` trait by calling back into Python evaluator methods (`evaluate_objective`, `evaluate_gradient`, `evaluate_constraints`, `evaluate_jacobian`, `evaluate_lagrangian_hessian`). Bounds and sparsity structure are cached in Rust-owned memory. The main solve loop releases the Python GIL via `py.allow_threads()`, and callbacks re-acquire it only for the duration of each evaluation:

```rust
let result = py.allow_threads(|| ripopt::solve(&problem, &opts));
```

This means the heavy linear algebra in the IPM iteration runs without GIL contention, while the JAX-compiled gradient and Hessian evaluations use Python callbacks through the GIL.

5.2.4 Licensing

Because ripopt is a source-level translation of Ipopt (EPL-2.0), it is distributed under the Eclipse Public License 2.0. EPL-2.0 is not viral: code that *depends on* ripopt does not need to adopt EPL-2.0, however, we have implemented a pure Python version in jax of the Ipopt algorithm in discopt, and have adopted the EPL-2.0 license for the entire discopt repository to avoid confusion and ensure that all components are covered by the same license.

5.3 Ipopt via cyipopt

The third backend uses the industry-standard Ipopt solver[10] via cyipopt Python bindings. This serves as the correctness reference: any discrepancy between the pure-JAX IPM or ripopt and Ipopt on a well-conditioned problem indicates a bug in the former. Ipopt's advantage is its mature sparse linear algebra (MUMPS, MA27/MA57) which scales better on large, sparse problems ($n > 1000$). Its disadvantage is the Fortran/C++ dependency chain and inability to batch or differentiate through the solve.

6 Convex Relaxation Framework

The quality of convex relaxations at B&B nodes directly determines solver efficiency. discopt provides a compositional relaxation compiler that walks the expression DAG and produces JIT-compatible relaxation functions.

6.1 McCormick Relaxation Compiler

The relaxation compiler implements 19 univariate and bivariate McCormick relaxation functions[11,12] covering all operations supported by the modeling API: bilinear products, addition, subtraction, negation, division, power, `exp`, square, `abs`, `sqrt`, `log`, `log2`, `log10`, `sin`, `cos`, `tan`, `sign`, `min`, and `max`.

Each function maps interval bounds $[x_l, x_u]$ and convex/concave enclosures (cv, cc) of the argument to tighter enclosures of the result. The compiler produces pure `jax.numpy` functions compatible with `jax.jit` and `jax.vmap`, enabling batch relaxation evaluation over multiple B&B nodes.

6.2 α BB Underestimators

For general twice-differentiable functions where McCormick relaxations are loose, the α BB method[13,14] adds a separable quadratic perturbation:

$$L_\alpha(x) = f(x) - \sum_{i=1}^{n} \alpha_i (x_i - x_i^l)(x_i^u - x_i) \tag{2}$$

The parameters α_i are chosen such that L_α is convex: $\alpha_i \geq \max(0, -\lambda_{\min}(\nabla^2 f)/2)$ where $\lambda_{\min}$ is the minimum eigenvalue of the Hessian. discopt provides two estimation methods:

- **Eigenvalue method**: exact computation via `jax.hessian` and `jnp.linalg.eigvalsh` at sampled points (accurate but requires $O(n^3)$ per sample)
- **Gershgorin method**: conservative estimate from Hessian diagonal dominance (faster, slightly looser bounds, zero soundness violations in testing)

The JAX-native α BB estimation uses `jax.vmap` over sample points, providing 10–100× speedup over the finite-difference fallback used for `.nl` file models.

6.3 Piecewise McCormick Relaxations

Piecewise McCormick[15] partitions the variable domain into k subintervals and constructs tighter McCormick envelopes on each partition. discopt supports both uniform partitioning and adaptive partitioning that concentrates breakpoints where $f''(x)$ is largest.

On representative test functions, piecewise McCormick with $k = 8$ partitions achieves significant gap reduction versus standard McCormick:

Function	Standard gap	Piecewise gap	Reduction
$\exp(x)$	0.999	0.066	93.4%
$\log(x)$	0.254	0.021	91.9%
x^2	2.664	0.167	93.7%
bilinear xy	varies	varies	¿30%

Table 2: Piecewise McCormick gap reduction with $k = 8$ partitions.

6.4 Convexity Detection

An expression DAG convexity detector classifies each subexpression as AFFINE, CONVEX, CONCAVE, or UNKNOWN using standard composition rules from convex analysis.[38] The key rules are:

- Constants, variables, and parameters are AFFINE.
- Sum of two CONVEX (resp. CONCAVE) expressions preserves curvature; CONVEX + CONCAVE yields UNKNOWN.
- A positive constant times a CONVEX expression is CONVEX; a negative constant flips curvature.
- exp(convex) is CONVEX (composition with nondecreasing convex outer function).
- log(concave) is CONCAVE (composition with nondecreasing concave outer function).
- x^{2k} (even integer power of affine) is CONVEX; x^p for $p > 1$ on nonneg domain is CONVEX; x^p for $0 < p < 1$ on nonneg domain is CONCAVE.
- Bilinear terms $x \cdot y$ are UNKNOWN.

Results are cached on expression IDs for efficiency. The detector runs in time linear in the DAG size and provides three downstream optimizations:

1. **Adaptive solver dispatch (E2):** When the entire model is classified as convex (convex objective for minimization, all constraints convex), the solver skips αBB convexification and McCormick relaxation compilation entirely. NLP solutions are directly valid lower bounds, eliminating relaxation overhead.

2. **Per-constraint OA cuts (E3):** In a nonconvex problem, individual constraints may still be convex. The convexity mask enables outer approximation cuts[20] on a per-constraint basis: tangent hyperplanes are globally valid underestimators for convex constraints, even when the overall problem is nonconvex. This replaces the earlier approach that only generated OA for affine (linear) constraints.

3. **Constraint classification:** A constraint $g(x) \leq 0$ is convex when g is CONVEX; $g(x) \geq 0$ is convex when g is CONCAVE; $g(x) = 0$ is convex only when g is AFFINE. The model-level classification checks the objective and all constraints simultaneously.

6.5 Specialized Envelopes

Standard McCormick relaxations compose relaxations through the expression tree, which can introduce composition error: outer functions see the relaxation of inner functions rather than the original function. Specialized envelopes compute exact convex/concave envelopes directly for common operations, bypassing this composition gap.

discopt provides the following envelope functions:

- `relax_power_int`(x, l, u, p): exact envelope for x^p with integer p. For even p, the function is convex everywhere, so the convex envelope is x^p itself and the concave envelope is the secant line. For odd $p \geq 3$, convexity/concavity depends on sign: convex on $[0, \infty)$, concave on $(-\infty, 0]$, and for mixed-sign domains a tangent line construction provides tight bounds.

- `relax_exp_tight`(x, l, u): convex envelope $= \exp(x)$, concave envelope = secant line from (l, e^l) to (u, e^u).

- `relax_log_tight`(x, l, u): concave envelope $= \log(x)$, convex envelope = secant line.

- `relax_exp_bilinear`, `relax_log_sum`, `relax_trilinear`, `relax_signomial`: envelopes for composite operations.

Tight trigonometric envelopes. For $\sin(x)$ on a bounded interval $[l, u]$, the relaxation quality depends on the curvature regime.[39] `relax_sin_tight` identifies three cases:

- *Concave regime* ($\sin'' < 0$, interval within $[2k\pi, (2k+1)\pi]$): cv = secant, cc $= \sin(x)$.

- *Convex regime* ($\sin'' > 0$, interval within $[(2k-1)\pi, 2k\pi]$): cv $= \sin(x)$, cc = secant.

- *Mixed or wide* ($\geq 2\pi$): falls back to $[-1, 1]$.

`relax_cos_tight` delegates via $\cos(x) = \sin(x + \pi/2)$. The compiler auto-dispatches these when the argument is a plain variable with known bounds.

Multivariate signomial envelopes. For signomial terms $\prod_{i=1}^{n} x_i^{a_i}$ with positive variables, `relax_signomial_multi` decomposes the product via logarithms:[40] $\prod x_i^{a_i} = \exp\left(\sum a_i \log x_i\right)$. Each $\log x_i$ is relaxed using tight log envelopes (concave), scaled by the exponent (sign-aware), summed, and composed with `exp` (convex). The compiler detects products of Variable^{Constant} terms and routes to this envelope when all lower bounds are positive.

Additional univariate envelopes. The envelope library also provides relaxations for six additional operations, each exploiting known curvature properties:

- `relax_asinh`: $\text{asinh}''(x) = -x/(1+x^2)^{3/2}$; concave on $[0, \infty)$, convex on $(-\infty, 0]$.
- `relax_acosh`: concave on $[1, \infty)$.
- `relax_atanh`: $\text{atanh}''(x) = 2x/(1-x^2)^2$; convex on $[0, 1)$, concave on $(-1, 0]$.
- `relax_erf`: $\text{erf}''(x) = -\frac{4x}{\sqrt{\pi}} e^{-x^2}$; convex on $(-\infty, 0]$, concave on $[0, \infty)$.
- `relax_log1p`: $\log(1+x)$ is concave on $(-1, \infty)$; numerically stable near $x = 0$.
- `relax_reciprocal`: $1/x$ is convex on $(0, \infty)$, concave on $(-\infty, 0)$.

For each, the convex regime uses function-value underestimator and secant overestimator, and vice versa for the concave regime. Mixed-sign domains use conservative min/max combinations.

The relaxation compiler automatically selects specialized envelopes when the expression matches a known pattern—for example, when x^p is encountered and x is a variable (or indexed variable) with known bounds, `relax_power_int` is used with the actual variable bounds from the B&B node rather than propagated interval bounds. This provides tighter relaxations because the envelope is exact for the monomial, whereas McCormick composition of $(x^2)^2$ for x^4 introduces two layers of relaxation error.

6.6 Learned Relaxations via ICNN

As an experimental feature, discopt supports learned convex relaxations using input convex neural networks (ICNNs).[16] An ICNN architecture with softplus non-negative weight constraints is trained to approximate tighter-than-McCormick underestimators for specific operations, with a soundness loss term that penalizes violations of the convexity and underestimation properties.

A `LearnedRelaxationRegistry` provides per-operation wrappers with automatic fallback to standard McCormick for unsupported operations. In testing, ICNN relaxations achieve approximately 10% gap reduction beyond standard McCormick, with the potential for 30%+ with more extensive training.

7 Differentiable Optimization

A distinguishing feature of discopt is the ability to differentiate through the optimization solve with respect to problem parameters. This enables decision-focused learning[31] where the model parameters (e.g., cost coefficients) are learned end-to-end from decision outcomes rather than prediction accuracy.

7.1 Level 1: Envelope Theorem

For a parametric NLP $\min_x f(x;p)$ subject to $g(x;p) \leq 0$, the envelope theorem gives:

$$\frac{df^*}{dp} = \left.\frac{\partial \mathcal{L}}{\partial p}\right|_{x^*,\lambda^*} = \left.\frac{\partial f}{\partial p}\right|_{x^*} + {\lambda^*}^T \left.\frac{\partial g}{\partial p}\right|_{x^*} \tag{3}$$

This requires no additional linear system solve—the optimal dual variables λ^* from the NLP solver directly provide the sensitivity. discopt implements this as a `DiffSolveResult.gradient(para` method that returns df^*/dp for any model parameter.

7.2 Level 3: Implicit Differentiation via KKT

For the full primal sensitivity matrix dx^*/dp, discopt differentiates the KKT conditions:

$$\begin{bmatrix} \nabla^2_{xx}\mathcal{L} & J_a^T \\ J_a & 0 \end{bmatrix} \begin{bmatrix} dx^*/dp \\ d\lambda^*/dp \end{bmatrix} = - \begin{bmatrix} \nabla^2_{xp}\mathcal{L} \\ \partial g_a/\partial p \end{bmatrix} \tag{4}$$

where J_a is the Jacobian of active constraints (including active variable bounds) and $\nabla^2_{xp}\mathcal{L}$ is the mixed Hessian of the Lagrangian. This is computed via `jax.hessian` and `jax.jacobian` applied to parametric DAG-compiled functions.

The `DiffSolveResultL3` class provides:

- `sensitivity_matrix()`: full dx^*/dp matrix
- `implicit_gradient(param)`: df^*/dp via the chain rule $\nabla_x f \cdot dx^*/dp + \partial f/\partial p$
- `approximate_resolve(new_params)`: first-order approximation $x^* + (dx^*/dp)\Delta p$
- `dual_sensitivity(param)`: $d\lambda^*/dp$ for constraint multiplier sensitivity
- `reduced_hessian()`: Hessian projected onto the null space of active constraints (for uncertainty quantification)
- `sensitivity(dp)`: fast re-query with cached KKT factorization (analogous to sIPOPT[28])

If the KKT system is ill-conditioned, discopt falls back to finite-perturbation smoothing, and then to L1 envelope theorem gradients.

7.3 Application: Decision-Focused Learning

The JAX-native differentiable solve is exposed via `jax.custom_jvp`, enabling it to be embedded in a larger JAX computation graph. A decision-focused learning pipeline:

1. A neural network $\hat{c} = \phi_\theta(x_{\text{features}})$ predicts cost parameters
2. The predicted costs are fed to `discopt.solve()` to obtain decisions $x^*(\hat{c})$
3. The true task loss $\ell(x^*(\hat{c}), c_{\text{true}})$ is computed

4. Gradients $\partial\ell/\partial\theta$ flow through the solver via L3 implicit differentiation

5. The network parameters θ are updated to minimize decision regret

On a resource allocation task with 3 activities, 5 features, and risk-averse objective, decision-focused learning with discopt achieves **36% regret reduction** compared to two-stage predict-then-optimize, despite having higher prediction MSE. This confirms the key insight: optimizing for decision quality beats optimizing for prediction accuracy.

On a mean-variance portfolio optimization task with 3 assets and nonlinear interactions, DFL similarly achieves lower regret and higher utility than the two-stage approach.

8 Computational Experiments

8.1 NLP Backend Comparison

We compare the three NLP backends on 5 test problems spanning unconstrained quadratics to constrained portfolio optimization. Table 3 reports objectives and wall-clock timing. All timings were collected on an Apple M4 Pro (CPU backend, JAX 0.8.2) and represent single-solve wall-clock time including model compilation.

Problem	f^*	IPM (s)	ripopt (s)	Ipopt (s)
Quadratic ($n = 2$)	0.000000	0.023	0.033	0.013
Constrained ($n = 5$)	1.250000	0.269	0.001	0.001
Rosenbrock ($n = 2$)	0.000000	0.590	0.082	0.114
Sum-of-squares ($n = 10$)	0.000000	1.189	0.049	0.070
Portfolio ($n = 8$)	0.002064	0.229	0.004	0.001
Total		2.30	0.17	0.20

Table 3: NLP backend comparison. All backends achieve identical objectives (to solver tolerance). Timings on Apple M4 Pro, CPU.

Key observations:

- All three backends produce **identical optimal objectives** to solver tolerance (10^{-6}), confirming correctness across the Rust, JAX, and C++ implementations.
- ripopt is competitive with Ipopt (total 0.17s vs 0.20s), and faster on Rosenbrock (0.082s vs 0.114s) and sum-of-squares (0.049s vs 0.070s).
- The pure-JAX IPM includes JIT compilation overhead on first evaluation; subsequent solves of similarly-structured problems benefit from cached JIT traces.
- On a quartic constrained NLP ($n = 10$), ripopt (0.334s) and the JAX IPM (0.795s) agree to 6.5×10^{-15} in objective value, confirming numerical equivalence.

8.2 MINLP Correctness

We validate on 24 MINLP problems (12 NLP + 12 MINLP with binary/integer variables) from the discopt test suite, with known optimal values:

- **24/24 solved correctly** with zero incorrect results
- Tolerances: absolute 10^{-4}, relative 10^{-3} for objectives; integrality tolerance 10^{-5}
- Problems include Rosenbrock, exponential, logarithmic, trigonometric, bilinear, and cardinality-constrained formulations

B&B convergence with the ripopt backend was verified on two MINLP instances: a simple 3-variable problem (optimal $f^* = 2.0$, 1 node, 0.039s) and a larger 6-variable formulation (optimal $f^* = 3.0$, 13 nodes, 2.38s).

8.2.1 MINLPLib Validation

We further validate on instances from MINLPLib,[41] imported via the `.nl` file parser. On 73 instances with ≤ 50 variables, discopt solves 49 to optimal/feasible status, with 18 of 27 checked instances matching known optima. Table 4 reports results on a representative 10-instance smoke test suite.

Instance	n	m	Status	f^*	Best Known	Time (s)	Nodes
chance	4	3	optimal	29.421	29.894	0.244	0
dispatch	4	2	optimal	3155.288	3155.288	0.258	0
ex1221	5	5	optimal	7.667	7.667	0.010	1
ex1226	5	5	optimal	-17.000	-17.000	0.176	3
gear	4	0	feasible	2.3e-11	0.000	60.000	25,153
nvs01	3	3	feasible	-4.904	12.470	60.731	7,787
nvs03	2	2	optimal	16.000	16.000	0.036	11
nvs04	2	0	optimal	0.720	0.720	0.078	9
nvs06	2	0	optimal	1.770	1.770	0.022	9

Table 4: MINLPLib smoke test results (10 instances, 60s time limit). Best known objectives from MINLPLib.

Of the 9 solvable instances, 7 reach optimal status (matching known optima to 10^{-3}), while 2 reach feasible solutions at the time limit. The `gear` instance finds a near-zero objective (2.3×10^{-11}, matching the known optimum 0.0) but cannot prove optimality without convex relaxations for the `.nl` import path. Instance `nvs01` reaches a local optimum for the same reason. The JAX computation accounts for 92–99.9% of solver time, with Rust tree management contributing $< 1\%$ and Python overhead $< 2\%$.

8.3 Relaxation Gap Reduction

Table 2 reports piecewise McCormick gap reduction. Additionally:

- α BB underestimators achieve zero soundness violations across 1000+ test points with both eigenvalue and Gershgorin methods
- ICNN learned relaxations provide approximately 10% additional gap reduction beyond standard McCormick with lightweight training
- The relaxation compiler supports mode switching between `standard`, `piecewise`, and `learned` at the solver level

8.4 Performance

- LP ($n = 100$): 3.5s $\rightarrow$ 0.20s warm (17$\times$ speedup)
- QP ($n = 100$): 65.7s $\rightarrow$ 0.29s warm (226$\times$ speedup)

9 Discussion

discopt demonstrates that a modern, multi-language MINLP solver can provide capabilities absent from existing monolithic solvers: differentiability, batch evaluation, and memory safety. Several limitations and directions for future work deserve discussion.

Convexity detection and adaptive dispatch. The expression DAG convexity detector implements the Disciplined Convex Programming (DCP) composition rules,[38] covering the operations supported by the modeling API. When a model is fully convex, the solver bypasses αBB convexification and McCormick relaxation compilation, relying instead on the NLP solution as a valid lower bound. For partially convex models, per-constraint classification enables outer approximation cuts on convex constraints while leaving nonconvex constraints to be handled by spatial branching. The current detector is conservative: expressions not covered by the composition rules are classified as UNKNOWN even if they happen to be convex, and the detector does not exploit second-order information (Hessian positive-semidefiniteness checks) for numerical verification.

Branching and bound tightening. Reliability branching[35] with LP-based strong branching provides a practical variable selection scheme that approaches full strong branching quality at amortized low cost. The pseudocost tracking in Rust provides per-variable observation counts that trigger strong branching only for unreliable variables. OBBT with incumbent cutoff[18] progressively tightens variable bounds throughout the tree, with each bound improvement propagating to all open nodes. The combination of reliability branching and incumbent-based OBBT addresses two of the key algorithmic gaps relative to BARON.[6]

Specialized envelopes versus compositional relaxations. The envelope functions for power, exponential, logarithmic, trigonometric, and signomial operations provide exact convex/concave envelopes, avoiding the composition error inherent in McCormick's recursive approach. The relaxation compiler automatically detects when a specialized envelope is applicable and routes accordingly. Recent additions include regime-aware sin/cos envelopes,[39]

multivariate signomial relaxations via logarithmic decomposition,[40] and envelopes for asinh, acosh, atanh, erf, $\log(1+x)$, and $1/x$ that exploit known curvature properties.

Limitations. The solver's performance on large-scale problems ($n > 1000$) is limited by the dense linear algebra in the pure-JAX IPM; the sparse IPM variant using lineax iterative solvers partially addresses this but lacks the numerical robustness of direct factorization methods. The GNN branching model currently runs with random weights; training on problem-specific data would improve variable selection. Head-to-head benchmarking against BARON on MINLPLib instances remains future work pending license acquisition.

10 Conclusion

We have presented discopt, a hybrid Rust/JAX/Python MINLP solver that bridges the gap between classical mathematical programming and modern differentiable programming. The solver provides three interchangeable NLP backends—a pure-JAX IPM, ripopt (a memory-safe Rust translation of Ipopt), and Ipopt via cyipopt—all producing identical results on benchmark problems. The differentiable solving capability enables decision-focused learning through nonconvex optimization, achieving 36% regret reduction on resource allocation tasks. The compositional relaxation compiler achieves 91–94% gap reduction via piecewise McCormick, and specialized envelopes for power, exponential, logarithmic, trigonometric, signomial, and inverse-hyperbolic operations provide exact convex/concave envelopes that avoid compositional relaxation error. An expression DAG convexity detector enables adaptive solver dispatch (skipping relaxation overhead for convex problems) and per-constraint outer approximation cuts. Reliability branching with pseudocost tracking and LP-based strong branching provides near-optimal variable selection, while OBBT with incumbent cutoff progressively tightens bounds throughout the search tree. The algebraic coefficient extraction path provides 17–226× speedups for structured subproblems. ripopt eliminates the Fortran/C++ dependency chain while preserving Ipopt's numerical robustness.

The system is open-source: discopt and ripopt are available under the EPL-2.0 License.

References

[1] Pietro Belotti, Christian Kirches, Sven Leyffer, Jeff Linderoth, James Luedtke, and Ashutosh Mahajan. Mixed-integer nonlinear optimization. *Acta Numerica*, 22:1–131, 2013. doi: 10.1017/S0962492913000032.

[2] Ignacio E. Grossmann. *Advanced Optimization for Process Systems Engineering*. Cambridge University Press, 2021. ISBN 978-1-107-18920-5. doi: 10.1017/9781108917834.

[3] Ailsa H. Land and Alison G. Doig. An automatic method of solving discrete programming problems. *Econometrica*, 28(3):497–520, 1960. doi: 10.2307/1910129.

[4] R. J. Dakin. A tree-search algorithm for mixed integer programming problems. *The Computer Journal*, 8(3):250–255, 1965. doi: 10.1093/comjnl/8.3.250.

[5] E. M. B. Smith and C. C. Pantelides. A symbolic reformulation/spatial branch-and-bound algorithm for the global optimisation of nonconvex MINLPs. *Computers & Chemical Engineering*, 23(4–5):457–478, 1999. doi: 10.1016/S0098-1354(98)00286-5.

[6] Mohit Tawarmalani and Nikolaos V. Sahinidis. A polyhedral branch-and-cut approach to global optimization. *Mathematical Programming*, 103(2):225–249, 2005. doi: 10.1007/s10107-005-0581-8.

[7] Ksenia Bestuzheva, Mathieu Besançon, Wei-Kun Chen, Antonia Chmiela, Tim Donkiewicz, Jasper van Doornmalen, Leon Eifler, Oliver Gaul, Gerald Gamrath, Ambros Gleixner, Leona Gottwald, Christoph Graczyk, Katrin Halbig, Alexander Hoen, Christopher Hojny, Rolf van der Hulst, Thorsten Koch, Marco Lübbecke, Stephen J. Maher, Frederic Matter, Erik Mühmer, Benjamin Müller, Marc E. Pfetsch, Daniel Rehfeldt, Steffan Schlein, Franziska Schlösser, Felipe Serrano, Yuji Shinano, Boro Sofranac, Mark Turner, Stefan Vigerske, Fabian Wegscheider, Philipp Wellner, Dieter Weninger, and Jakob Witzig. Enabling research through the SCIP optimization suite 8.0. *ACM Transactions on Mathematical Software*, 49(2):1–21, 2023. doi: 10.1145/3585516.

[8] Pierre Bonami, Lorenz T. Biegler, Andrew R. Conn, Gérard Cornuéjols, Ignacio E. Grossmann, Carl D. Laird, Jon Lee, Andrea Lodi, François Margot, Nicolas Sawaya, and Andreas Wächter. An algorithmic framework for convex mixed integer nonlinear programs. *Discrete Optimization*, 5(2):186–204, 2008. doi: 10.1016/j.disopt.2006.10.011.

[9] Sanjay Mehrotra. On the implementation of a primal-dual interior point method. *SIAM Journal on Optimization*, 2(4):575–601, 1992. doi: 10.1137/0802028.

[10] Andreas Wächter and Lorenz T. Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical Programming*, 106(1):25–57, 2006. doi: 10.1007/s10107-004-0559-y.

[11] Garth P. McCormick. Computability of global solutions to factorable nonconvex programs: Part I — Convex underestimating problems. *Mathematical Programming*, 10(1):147–175, 1976. doi: 10.1007/BF01580665.

[12] Angelos Tsoukalas and Alexander Mitsos. Multivariate McCormick relaxations. *Journal of Global Optimization*, 59(2–3):633–662, 2014. doi: 10.1007/s10898-014-0176-0.

[13] Ioannis P. Androulakis, Costas D. Maranas, and Christodoulos A. Floudas. αBB: A global optimization method for general constrained nonconvex problems. *Journal of Global Optimization*, 7(4):337–363, 1995. doi: 10.1007/BF01099647.

[14] Claire S. Adjiman, Ioannis P. Androulakis, and Christodoulos A. Floudas. A global optimization method, αBB, for general twice-differentiable constrained NLPs — II. Implementation and computational results. *Computers & Chemical Engineering*, 22(9):1159–1179, 1998. doi: 10.1016/S0098-1354(98)00218-X.

[15] Pedro M. Castro. Tightening piecewise McCormick relaxations for bilinear problems. *Computers & Chemical Engineering*, 72:300–311, 2015. doi: 10.1016/j.compchemeng.2014.03.025.

[16] Brandon Amos, Lei Xu, and J. Zico Kolter. Input convex neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 146–155, 2017. URL https://proceedings.mlr.press/v70/amos17b.html.

[17] Pietro Belotti, Jon Lee, Leo Liberti, François Margot, and Andreas Wächter. Branching and bounds tightening techniques for non-convex MINLP. *Optimization Methods and Software*, 24(4–5):597–634, 2009. doi: 10.1080/10556780903087124.

[18] Ambros M. Gleixner, Timo Berthold, Benjamin Müller, and Stefan Weltge. Three enhancements for optimization-based bound tightening. *Journal of Global Optimization*, 67(4):731–757, 2017. doi: 10.1007/s10898-016-0450-4.

[19] Matteo Fischetti, Fred Glover, and Andrea Lodi. The feasibility pump. *Mathematical Programming*, 104(1):91–104, 2005. doi: 10.1007/s10107-004-0570-3.

[20] Marco A. Duran and Ignacio E. Grossmann. An outer-approximation algorithm for a class of mixed-integer nonlinear programs. *Mathematical Programming*, 36(3):307–339, 1986. doi: 10.1007/BF02592064.

[21] Roger Fletcher and Sven Leyffer. Solving mixed integer nonlinear programs by outer approximation. *Mathematical Programming*, 66(1–3):327–349, 1994. doi: 10.1007/BF01581153.

[22] Hanif D. Sherali and Warren P. Adams. A hierarchy of relaxations between the continuous and convex hull representations for zero-one programming problems. *SIAM Journal on Discrete Mathematics*, 3(3):411–430, 1990. doi: 10.1137/0403036.

[23] Stephen J. Wright. *Primal-Dual Interior-Point Methods*. SIAM, 1997. ISBN 978-0-89871-382-4. doi: 10.1137/1.9781611971453.

[24] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006. doi: 10.1007/978-0-387-40065-5.

[25] Andreas Wächter and Lorenz T. Biegler. Line search filter methods for nonlinear programming: Motivation and global convergence. *SIAM Journal on Optimization*, 16(1): 1–31, 2005. doi: 10.1137/S1052623403426556.

[26] Haihao Lu, Zedong Peng, and Jinwen Yang. MPAX: Mathematical programming in JAX. *arXiv preprint arXiv:2412.09734*, 2024. URL https://arxiv.org/abs/2412.09734. First-order LP/QP solver natively in JAX with GPU/TPU portability and batched solving.

[27] Anthony V. Fiacco. *Introduction to Sensitivity and Stability Analysis in Nonlinear Programming*. Academic Press, 1983. ISBN 978-0-12-254450-7.

[28] Hans Pirnay, Rodrigo López-Negrete, and Lorenz T. Biegler. Optimal sensitivity based on IPOPT. *Mathematical Programming Computation*, 4(4):307–331, 2012. doi: 10.1007/s12532-012-0043-2.

[29] Brandon Amos and J. Zico Kolter. OptNet: Differentiable optimization as a layer in neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 136–145, 2017. URL https://proceedings.mlr.press/v70/amos17a.html.

[30] Akshay Agrawal, Brandon Amos, Shane Barratt, Stephen Boyd, Steven Diamond, and J. Zico Kolter. Differentiable convex optimization layers. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019. URL https://proceedings.neurips.cc/paper/2019/hash/9ce3c52fc54362e22053399d3181c638-Abstract.html.

[31] Adam N. Elmachtoub and Paul Grigas. Smart "Predict, then Optimize". *Management Science*, 68(1):9–26, 2022. doi: 10.1287/mnsc.2020.3922.

[32] Bryan Wilder, Bistra Dilkina, and Milind Tambe. Melding the data-decisions pipeline: Decision-focused learning for combinatorial optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 1658–1665, 2019. doi: 10.1609/aaai.v33i01.33011658.

[33] Jayanta Mandi, Emir Demirović, Peter J. Stuckey, and Tias Guns. Smart predict-and-optimize for hard combinatorial optimization problems. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 1603–1610, 2020. doi: 10.1609/aaai.v34i02.5521.

[34] PyO3 Contributors. PyO3: Rust bindings for the Python interpreter, 2024. URL https://github.com/PyO3/pyo3.

[35] Tobias Achterberg, Thorsten Koch, and Alexander Martin. Branching rules revisited. *Operations Research Letters*, 33(1):42–54, 2005. doi: 10.1016/j.orl.2004.04.002.

[36] Q. Huangfu and J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018. doi: 10.1007/s12532-017-0130-5.

[37] Hong S. Ryoo and Nikolaos V. Sahinidis. A branch-and-reduce approach to global optimization. *Journal of Global Optimization*, 8(2):107–138, 1996. doi: 10.1007/BF00138689.

[38] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. ISBN 978-0-521-83378-3. doi: 10.1017/CBO9780511804441.

[39] Marco Locatelli and Fabio Schoen. *Global Optimization: Theory, Algorithms, and Applications*. SIAM, 2014. ISBN 978-1-611972-66-5. doi: 10.1137/1.9781611972672.

[40] Costas D. Maranas and Christodoulos A. Floudas. Global optimization in generalized geometric programming. *Computers & Chemical Engineering*, 21(4):351–369, 1997. doi: 10.1016/S0098-1354(96)00282-7.

[41] Michael R. Bussieck, Arne Stolbjerg Drud, and Alexander Meeraus. MINLPLib — A collection of test models for mixed-integer nonlinear programming. *INFORMS Journal on Computing*, 15(1):114–119, 2003. doi: 10.1287/ijoc.15.1.114.15159.